General Developer Journey		Points	4	3	2	1	0	Score	Min Score	Max Score	Comment	Found issues
Phase	**Action**	**Description**	**Great Experience**	**Good Experience**	**Neutral Experience**	**Bad Experience**	**Terrible Experience**					
Phase 1: Environment Setup & Sanity Check												
Development Environment Setup	Install TT-NN and run basic examples		One-click setup with containerized environments, automatic dependency resolution, and integrated development tools	Comprehensive setup guide with automated scripts, clear dependency management, and troubleshooting sections.	Standard setup documentation with most dependencies covered, but some manual configuration still required for optimal performance.	Basic setup instructions exist but are outdated or incomplete, leading to version conflicts and missing dependencies.	No setup documentation. Developers must figure out dependencies, versions, and configuration from scattered sources.	4	0	4	https://github.com/xkuze/Tenstorrent	
	Verify Installation	Run basic operations tutorial	Script runs first try	Works after minor fixes	Several attempts needed	Multiple errors encountered	Cannot get basic script working	4	0	4		
Phase 2: Implementation												
Tensor Layout and Memory Management	Understanding TT-NN's tensor layouts		Intuitive interface that automatically suggests optimal layouts based on operation types, with comprehensive performance benchmarks.	Clear documentation with performance guidelines and multiple examples showing layout choices for different operation types.	Documentation covers the basics with some examples, but you still need significant experimentation to optimize for their specific use case.	Basic documentation exists but lacks practical examples.The options are listed but it is not clear how they affect performance and use cases.	No clear documentation on when to use ROW_MAJOR vs TILE. Forced to reverse-engineer from source code or trial-and-error, leading to longer than expected experimentation based on error messages.	3	0	4	Needed experimentation with TILE_LAYOUT	
	Managing tensor shapes and padding		Automatic reshaping and padding with performance warnings when suboptimal choices are made.	Clear documentation with best padding choices explained and helpful error messages that suggest corrections.	Basic padding requirements are documented, but optimal strategies for performance are unclear.	Some documentation exists but is scattered. Error messages are unclear, such as, "Invalid tensor shape", but without guidance on how to fix.	Padding requirements are undocumented, leading to runtime crashes with no clear error messages. Excessive time is wasted figuring out alignment requirements.	3	0	4	Reshape needed for tile alignment, Shape mismatch: masks [B,H,W] vs output [B,1,H,W]	
	Memory allocation		Automatic memory management.	There is clear documentation on data allocation and movement, with tradeoffs explained in sufficient detail.	Basic data movement is documented, and there are enough examples to get by, but choices are not explained.	Basic memory management must be managed manually with minimal guidance or feedback.	No visibility into memory usage, frequent out-of-memory crashes with no guidance on optimization strategies.	4	0	4		
Framework Translation Challenges	Converting from PyTorch/TensorFlow patterns		Migration is straightforward and easy using similar or identical flows	Comprehensive migration guides for common cases.	Decent operation mapping with some pattern examples, but gaps exist for advanced use cases.	Basic operation mapping exists but complex patterns (like custom layers) have no guidance for translation.	No migration guides or equivalent operation mappings. Conversion from scratch with completely different paradigms.	4	0	4	Hybrid approach - Conv2d on CPU, torchmetrics.Dice not found, used F1Score instead	

Model Conversion Workflow	Handling unsupported operations	Automatic decomposition of unsupported operations with performance warnings and optimization suggestions.	Comprehensive operation support with detailed decomposition guides and performance implications.	Clear support matrix with basic decomposition examples for common unsupported operations.	Supported operations are listed but decomposition strategies for unsupported ones are undocumented.	No clear list of supported vs unsupported operations. Discover limitations at runtime with no workaround guidance.	4	0	4	sed matmul for 1x1 conv, Full Conv2d UNet not straightforward on TT-NN
	PyTorch to TT-NN conversion sequence	Conversion for most models is straightforward with intelligent handling of edge cases and clear reporting of any manual steps needed.	Well-documented conversion pipeline with for common architectures and clear guidance for edge cases.	Standard workflow exists with examples, but complex models require significant manual intervention.	Basic conversion steps documented but many edge cases and are not, leading to frequent failures.	No established workflow. Explore entire conversion pipeline from scratch, often discovering incompatibilities late in the process.	4	0	4	UNet required hybrid approach, Checkpoint format incompatibility (Lightning vs torch.save)
	Weight conversion and initialization	Automatic weight conversion with integrity checking and performance optimization during conversion.	Comprehensive weight conversion within operations and support for all major formats.	Conversion relatively straightforward with most common formats, but some manual work required for complex cases.	Poorly documented and not clear how to handle all PyTorch weight formats.	Developers must manually handle format conversions with no validation of correctness.	4	0	4	ttnn.from_torch bfloat16 worked
	Input preprocessing pipelines	Automatic input preprocessing with optimal performance.	Comprehensive preprocessing with easy integration patterns and performance optimization.	Standard preprocessing operations supported with some integration examples.	Basic preprocessing operations available but integration with existing pipelines is unclear.	No guidance on how to handle preprocessing. All preprocessing must be done from scratch in TT-NN format.	4	0	4	Standard PyTorch preprocessing
API Learning Curve	Understanding TT-NN programming approach	Interactive tutorials and clear conceptual bridges that make the transition intuitive for developers from other frameworks.	Comprehensive guide comparing approaches with clear migration patterns and best practices.	Decent explanation of functional approach with some comparative examples to other frameworks.	Basic functional concepts explained but practical application to deep learning workflows is unclear.	No conceptual documentation. PyTorch developers are completely lost trying to understand the functional paradigm without guidance.	4	0	4	Functional approach OK
	Learning function signatures and parameters	TT-NN API matches PyTorch with near-identical usage patterns.	Comprehensive API documentation with detailed examples and parameter impact explanations.	Standard API documentation with basic examples, but complex parameter interactions are unclear.	Basic API reference exists but lacks examples and parameter descriptions are minimal.	No API documentation. It is necessary to read source code to understand function signatures and parameter meanings.	4	0	4	matmul, add documented, x.T deprecation warning
	Error handling and edge cases	Intelligent error handling with automatic recovery suggestions and proactive edge case detection.	Comprehensive error handling guide with clear recovery patterns and prevention strategies.	Standard error handling patterns with some examples, but edge case handling is inconsistent.	Basic error types documented but recovery strategies and prevention methods are unclear.	No error handling guidance. Runtime failures provide no context about what went wrong or how to fix it.		0	4	

Integration and Deployment	Device initialization and context management		Automatic device management with intelligent resource allocation and cleanup.	Straighforward device management with clear and comprehensive documentation.	Standard initialization patterns documented with basic context management, but advanced scenarios lack guidance.	Basic device initialization examples exist but documentation is lacking and more complex scnerios are not explained.	No guidance on device management.	4	0	4	open_device(device_id=2) worked
	Inference loops and batch processing		Intelligent batching with automatic optimization based on model characteristics and hardware capabilities, plus real-time performance feedback.	Comprehensive inference guide with multiple batching strategies, performance benchmarks, and optimization recommendations.	Standard inference patterns with some batching examples, but optimization for throughput/latency tradeoffs requires experimentation.	Basic inference examples exist but batching strategies and performance considerations are undocumented.	No examples of proper inference patterns. Experiment with batching, memory management, and performance optimization completely from scratch.	4	0	4	Basic inference, batch=4
Validation	Correctness	Compare outputs using PCC	PCC > 0.99 immediately	PCC > 0.99 after minor fixes	PCC acceptable with some work	PCC marginal, concerning	PCC unacceptable	4	0	4	CC = 0.999976 > 0.99
Phase 3: Debugging and Performance											
Debugging	Using tt-smi to manage and troubleshoot hardware		Good experience + in case of hardware trouble: uses tt-smi with success, such as resetting boards	Understands how tt-smi works and ran it enough times to get familiar with it	Having to deal with e.g. documentation to comprehend the tool's capabilities, potential unforeseen problems with the tool itself	tt-smi doesn't work as expected and no sources help in some cases	Problems so fundamental, such as no hasrdware detected, that completely prevent the developer from using the tool or the tool crashes the hardware		0	4	
	Engaging watcher to detect and act against hangs		Good experience + having watcher work as intended in case of trouble	Understands how watcher works and ran it enough times to get familiar with it	Having to deal with e.g. documentation to comprehend the tool's capabilities, potential unforeseen problems with the tool itself	watcher doesn't work as expected and no sources help in some cases	Problems so fundamental, such as no hardware detected, that completely prevent the developer from using the tool or the tool crashes the hardware		0	4	
	Debugging tools and error messages		Excellent error messages with automatic suggestions for fixes. Advanced debugging with step-through capabilities and tensor visualization.	Clear error messages with actionable suggestions. Good debugging tools with tensor inspection capabilities.	Reasonable error messages with some context. Basic profiling tools available but require manual interpretation.	Basic error messages with minimal context. Simple debugging tools that provide limited insight into execution flow.	Cryptic error messages with no stack traces or context. No debugging utilities available.		0	4	
Performance	Performance bottleneck identification		Real-time performance monitoring with automatic bottleneck detection and specific optimization suggestions.	Detailed performance profiling with clear bottleneck identification and optimization recommendations.	Operation-level timing with some baseline comparisons, but optimization strategies are unclear.	Basic timing information available but no guidance on what constitutes good vs bad performance for specific operations.	No performance visibility. Developers can only measure end-to-end time with no insight into which operations are underperforming.		0	4	

	Influence of scheduling on a mesh of cores on total runtime	Seeing how core dispatchment model may affect overall multi-core performance	Good experience + experimenting with success on their own	Seeing good scale-up accordingly as amounts of data and cores utilised increase	Potential unforeseen warnings or mishaps as data/cores increase	No scale-up noticed or had significant problems getting any performance gain	No scale-up at all, silent/persistent problems	0	4
Phase 4: Overall assessment									
Summary	Overall impact assessment		**Prototyping and deployment is quick and easy to achieve. Default implementation produces good performance.**	**Solutions can be implemented efficiently with clear guidance, spending most time on actual model development rather than framework learning.**	**Complete the project with reasonable effort, but require significant learning time and some experimentation.**	**Eventually succeed but only after extensive trial-and-error, taking significantly onger than expected with high frustration levels.**	**Task is abandoned after significant frustration, finally being unable to get basic functionality working.**	0	4

TT-NN Visualizer

Phase	Action	Description	Great Experience	Good Experience	Neutral Experience	Bad Experience	Terrible Experience		
Phase 1: Environment Setup & Sanity Check									
Time Taken: How long did this phase take? Installation Issues: Did pip install work smoothly? Any driver/dependency errors? Sanity Check: Did ttnn_basic_operations.py run successfully first try? Clarity Score (1-5): How clear were initial setup instructions?	Install tt-metal with profiler and ttnn	Clone tt-metal repository, build tt-metal with profiler enabled, build ttnn from source to access built tt-metal	Instructions work smoothly	Minor dependency issues resolved	Some trial and error needed	Installation fails multiple times	Complete installation failure	0	4
	Verify Installation	Run basic operations tutorial	Script runs first try	Works after minor fixes	Several attempts needed	Multiple errors encountered	Cannot get basic script working	0	4
Phase 2: TTNN-Visualizer installation and verification									
Time Taken: How long did this phase take? Installation Issues: Did pip install work smoothly? Any driver/dependency errors? Sanity Check: Did ttnn-visualizer run successfully first try (opened website)? Clarity Score (1-5): How clear were initial setup instructions?	Install ttnn-visualizer	pip install ttnn-visualizer	Instructions work smoothly	Minor dependency issues resolved	Some trial and error needed	Installation fails multiple times	Complete installation failure	0	4
	Run ttnn-visualizer	ttnn-visualizer	Script runs first try - homepage shows up	Works after minor fixes	Several attempts needed	Multiple errors encountered	Cannot get basic script working	0	4
Phase 3: Profiling and gathering memory report data									
Profiling Success: Did profiling complete successfully without crashing or hanging? Clarity of Profiling Instructions (1–5): How clear and understandable were the steps to collect profiling and memory data? Sanity Check: Did the profiling output get generated and look reasonable on the first attempt? Data Quality: Was the output data detailed and accurate enough to be useful? Visualization Integration: Were the profiling results easy to visualize or import into tools like TTNN Visualizer? Error Feedback: Were error messages clear and actionable if profiling or reporting failed?	Set configuration file	Create a setup file and set the global variable to point to it	Config file loads correctly and profiler can start	Minor fixes needed to recognize config	Needs debugging to activate correctly	Profiler fails to recognize config	Config setup consistently fails	0	4
	Gather data	Collect memory data by running the target model with profiling configuration	Memory data is generated as expected	Minor issues when generating data	Data generated after several runs	Multiple profiling errors	Data not generated at all	0	4
	Collect created report	Collect created data from machine	File collected without issue	Minor directory confusion resolved	Took multiple attempts to locate	File not found at first	Could not locate report file	0	4
	View memory report	Upload collected data to the visualizer under the Memory reports section.	Upload successful, data displays correctly	Several tries needed	Needs re-export to work	Upload fails initially	Upload not possible	0	4
Phase 4: Profiling and gathering performance report data									
Profiling Success: Did profiling complete successfully without crashing or hanging? Clarity of Profiling Instructions (1–5): How clear and understandable were the steps to collect profiling and performance data? Sanity Check: Did the profiling output get generated and look reasonable on the first attempt? Report Usefulness: Was the report detailed and useful for understanding runtime performance (e.g., op latency, throughput)? Visualization Integration: Were the profiling results easy to visualize or import into tools like TTNN Visualizer? Error Feedback: Were error messages clear and actionable if profiling or reporting failed?	Gather performance data	Collect performance data by running model with Tracy profiler enabled	Tracy data collected as expected	Tracy data collected after config tweaks	Needed reruns to collect data	Multiple errors in profiler	No data collected	0	4
	Collect created report	Collect created data from machine	File collected without issue	Minor directory confusion resolved	Took multiple attempts to locate	File not found at first	Could not locate report file	0	4
	View performance memory report	Upload collected data to the visualizer under the Performance reports section.	Upload successful and data visualized	Minor fix to display data	Needs re-export to work	Viewer crashes or fails	Upload not possible	0	4
Phase 5: Analyze results									

	Review Memory & Performance Reports	Explore visualizations to identify performance bottlenecks and data movement issues	Clear bottlenecks identified, leads to optimization ideas	Some areas of concern identified	Needed deeper manual analysis	Difficult to interpret data	Visualization unclear or broken	0	4
Analysis Time: How long did it take to interpret the profiling and memory/performance data? Insight Clarity: Were key insights (e.g., bottlenecks, memory hotspots, slow ops) easy to identify? Data Visualization Quality: Were visualizations (graphs, tables) effective in presenting the results? Model Understanding: Did the analysis help you better understand the model's execution and dataflow? Guidance Quality: Were the analysis tools or documentation helpful in suggesting next steps or optimizations? Customization: Could you easily filter or drill down into specific parts of the data (e.g., a single op or layer)? Overall Usefulness (1–5): How useful was this analysis phase in preparing for optimization or debugging?	Generate insights	document observations about hotspots, memory usage patterns, layout problems, etc.	Actionable insights recorded and shared	General insights gathered	Some observations made, needs more work	Insights unclear	No useful information derived	0	4

TT-NN Tracer

Phase 1: get all ops with their parameters using the tracer tool	Action	Description	Great Experience	Good Experience	Neutral Experience	Bad Experience	Terrible Experience		
extract all the ops in the torch model along with the needed configurations of it to create python unit tests that will compute the PCC between torch and ttnn outputs for each op	extract all the ops in the torch model	extract all the ops in the torch model along with the needed configurations of it to create python unit tests that will compute the PCC between torch and ttnn outputs for each op python sample_tracer.py --model model_name --input-shape b c h w	README instructions were clear and I could get started with the tool immediately	I had to install a few additonal libraries but was able to do and use the tool within no time	Needed sometime to run the tool but instrucions were clear enough to help	Even after following the instructions it took me a long time to use the tool	Instructions not clear at all and could not use the tool	0	4
Phase 2: auto-generate the op-level unit tests for different configurations of the op			Script runs first try	Works after minor fixes	Several attempts needed	Multiple errors encountered	Cannot get basic script working	0	4
use the tool for automatic generation and run of the op-level unit tests	generate unit tests	generate unit tests	The generated pytests are extremely useful, I did not had to write even a single test manually to test on TTNN	I found them helpful but needed to add some more test cases to help complete coverage	Found them helpful, it missed a few minor edge cases that had to be added manually	Did not find them helpful, I had to add a lot of test cases manually and missed a lot of edge cases	Not at all useful, I had to rewrite or add all the test cases and edge cases manually to test on TTNN	0	4
Phase 3: use the tool to generate the full model graph + TTNN metadata									
Full automation of this step is still in progress. Semi-automated step generates a draft of ttnn ops required to run model, along with the pytest	Generate full model graph	generate model graph	All the files and meta data generated by the tool has been very helpful, helped me speed up my development on TTNN	Found it helpful, gave me a good idea of the model and could take next steps quickly because of the generated files	Good to have as a reference for a quick look-up when doing the development on TTNN	Did not find it that helpful, nothing that would help speed up the development	Not at all helpful, the files generated does not have any relevant information that helps me speed-up my development on TTNN	0	4
Phase 4: add unsupported ops for auto-generation of unit tests									

add missing ops support for auto generation of unit tests	add missing ops	add missing ops	Easy to understand the README file, clearly understood what need to be done to add the unsupported ops, added them and the tool generated the pytests for the added ops	README file was easy to understand, found it helpufl to add new ops, took sometime but was able to add the new ops and the tool auto-generated the pytests	README was helpful, but took me sometime to add the new supported ops, I anyways had to write a sample pytest to make it work once, it took me longer than anticipated but was able to add support for new ops, generate and use the pytest to test out the newly added ops.	README not helpful, adding new ops looks very complicated, had to struggle to add support for new ops, took a long time to add any support for new ops	README not at all useful, could not understand how to add support for new ops in the tool, could not get it working		0	4